# Field Service Management Backend Beginner Guide

## Goal

```
Build the backend step by step using ASP.NET Core Web API, Entity Framework Core, SQL Server, Code F
```

## Project Structure

```
Backend/
 FieldServiceManagement.API/
   Controllers/
   Models/
   Data/
   Migrations/
   Program.cs
   appsettings.json
```

## NuGet Packages

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

## Connection String

```
{
  "ConnectionStrings": {
    "DefaultConnection":"Server=localhost,1433;Database=FieldServiceDB;User Id=SA;Password=YOUR_PASS
  }
}
```

## User Model

```
namespace FieldServiceManagement.API.Models;

public class User
{
    public int Id { get; set; }
    public string FirstName { get; set; } = "";
    public string LastName { get; set; } = "";
    public string Email { get; set; } = "";
    public string PhoneNumber { get; set; } = "";
    public string Role { get; set; } = "";
}
```

## AppDbContext

```
using Microsoft.EntityFrameworkCore;
using FieldServiceManagement.API.Models;

namespace FieldServiceManagement.API.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options):base(options){}
    public DbSet<User> Users { get; set; }
}
```

## Program.cs Registration

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

## UsersController

```
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly AppDbContext _context;
    public UsersController(AppDbContext context){ _context=context; }

    [HttpGet]
    public async Task<IActionResult> GetAllUsers()
    {
        var users=await _context.Users.ToListAsync();
        return Ok(users);
    }

    [HttpPost]
    public async Task<IActionResult> InsertUser(User user)
    {
        _context.Users.Add(user);
        await _context.SaveChangesAsync();
        return Ok("User Added Successfully");
    }
}
```

## EF Core Commands

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

## Testing in Scalar

```
GET /api/users -> should return [] initially.

POST /api/users

{
  "firstName":"Aniruddh",
  "lastName":"Parmar",
  "email":"aniruddh@gmail.com",
  "phoneNumber":"9876543210",
  "role":"Owner"
}

Expected response:
User Added Successfully

Then call GET /api/users again to verify the inserted record.
```

## Verify in SQL Server

```
Run:
SELECT * FROM Users;
```

## Concepts Learned

- Model (Entity)
- DbContext
- DbSet
- Dependency Injection
- Controller
- GET API
- POST API
- Entity Framework Core
- Code First
- Migration
- Database Update
- Scalar API Testing

## Next Roadmap

1. Authentication (JWT)
2. Customer
3. Technician

4. Services
5. Service Requests
6. Work Reports
7. Invoices
8. Notifications
9. Status History
10. React API Integration